

Introdução ao GNU Octave para Cálculo Numérico

Sumário

1. Introdução	3
2. Fundamentos de Programação (conceitos transferíveis)	4
2.1. Variáveis, tipos e operações aritméticas	4
2.2. Entrada e saída de dados (I/O)	4
2.3. Estruturas de controle: if/else, for, while	5
2.4. Vetores e matrizes	7
3. Scripts e Funções	9
3.1. Scripts (.m)	9
3.2. Funções (modularização)	10
4. Operações matriciais e vetoriais no Octave	12
4.1. Transposta, multiplicação, inversão	12
4.2. Indexação e slicing	13
4.3. Operações elemento a elemento	13
5. Gráficos 2D e 3D (visualização de resultados)	15
5.1. Gráficos 2D - Plotar funções e dados	15
5.2. Gráficos 3D — Visualizar superfícies	16
6. Representação numérica (binário, hexadecimal e decimal)	17
6.1. Conversões (binário/hex/decimal) no Octave	17
6.2. Operações com números binários (inteiro)	17
6.3. Binário “com parte decimal” (frações binárias)	18
6.4. Complemento de 2 (inteiros com sinal)	19
6.5. Tipos inteiros: int8, int16, int32, int64	20
6.6. Representação de números decimais (ponto flutuante)	21
6.7. Tabela ASCII — exemplos essenciais	22
7. Representação simbólica de equações no Octave	24
7.1. Introdução ao pacote symbolic	24
7.2. Criar variáveis simbólicas	24
7.3. Criar e manipular equações simbólicas	24

7.4.	Operações com expressões simbólicas	25
7.5.	Derivadas e integrais simbólicas	26
7.6.	Substituição de valores em expressões simbólicas	27
8.	Octave aplicado ao Cálculo Numérico	28
8.1.	Sistemas lineares com $A \setminus b$ (exemplo da engenharia)	28
8.2	Exploração de condicionamento (sensibilidade a perturbações)	28

1.Introdução

O que é o Octave e por que usar

O GNU Octave é um ambiente livre (open-source) muito usado para cálculo numérico, com linguagem compatível com MATLAB em grande parte dos comandos.

Onde ele ajuda na Engenharia:

- Sistemas lineares e álgebra linear (correntes em circuitos, equilíbrio estático, malhas).
- Ajuste de curvas e interpolação (dados experimentais, sensores, viscosidade vs. temperatura).
- Simulação e visualização (respostas no tempo, gráficos, comparação de modelos).

Como executar comandos:

- No prompt interativo (linha a linha).
- Em scripts (.m) para automatizar.
- Em funções (function ... endfunction) para reutilizar.

Ajuda e documentação

Use estes comandos sempre que esquecer uma sintaxe:

- help comando
- doc comando (quando disponível)
- lookfor palavra

Exemplo **octave**:

```
help linspace
lookfor matrix
help linspace
lookfor matrix
```

Limpar e organizar a sessão

Todo script de engenharia deve começar com a limpeza do ambiente para evitar que variáveis de cálculos anteriores interfiram nos novos.

```
% --- Início Padrão de Script ---
clear; % Apaga todas as variáveis da memória
clc;   % Limpa a tela do console (Command Window)
close all; % Fecha todas as janelas de gráficos abertas
% -----
```

2. Fundamentos de Programação (conceitos transferíveis)

2.1. Variáveis, tipos e operações aritméticas

No Octave, variáveis são criadas por atribuição simples. Não é necessário declarar tipo — o Octave infere automaticamente.

```
% Exemplo 1: Operações básicas com números reais
a = 10;           % Atribui 10 à variável 'a'
b = 3;           % Atribui 3 à variável 'b'
soma = a + b      % Calcula a soma (resultado: 13)
produto = a * b   % Calcula o produto (resultado: 30)
divisao = a / b   % Calcula a divisão (resultado: 3.3333...)
potencia = a^2    % Calcula a potência (resultado: 100)
```

Pontos importantes

- `;` no final **suprime a saída** (não mostra o resultado na tela).
- Sem `;`, o Octave exibe o resultado automaticamente.
- Octave usa, por padrão, números em **ponto flutuante** (tipo **double**, 64 bits).

Exemplo com e sem ponto-e-vírgula:

```
x = 5; % Não mostra nada
y = 5 % Mostra: y = 5
```

Ajuste de formato de exibição:

```
% Mostra números com menos casas decimais
format short
pi           % Resultado: 3.1416

% Mostra números com mais casas decimais
format long
pi           % Resultado: 3.14159265358979

% Volta ao padrão
format default
```

2.2. Entrada e saída de dados (I/O)

Entrada simples `input()`:

```
% Solicita ao usuário que digite um valor
x = input("Digite o valor de x: ");

% Mostra o valor digitado com formatação
fprintf("Você digitou x = %.4f\n", x);
% %.4f = mostra 4 casas decimais
% \n = quebra de linha
```

Exemplo Prático

```
% Entrada
raio = input("Digite o raio do círculo (em cm): ");

% Cálculo
area = pi * raio^2;
perimetro = 2 * pi * raio;

% Saída formatada
fprintf("Raio: %.2f cm\n", raio);
fprintf("Área: %.2f cm²\n", area);
fprintf("Perímetro: %.2f cm\n", perimetro);
```

Função `disp()` para exibição simples:

```
% Exibe texto ou valor
disp("Olá, mundo!");
disp(42);
disp([1 2 3]); % Exibe um vetor
```

2.3. Estruturas de controle: if/else, for, while

if/ elseif/else

```
% Exemplo: Classificar um número
x = -2;

if x > 0
    % Se x é positivo
    disp("x é positivo");
elseif x == 0
    % Se x é zero
    disp("x é zero");
else
    % Se x é negativo
    disp("x é negativo");
end

% Resultado: x é negativo
```

Operadores de comparação:

- == (igual)
- ~= (diferente)
- > (maior)
- < (menor)
- >= (maior ou igual)
- <= (menor ou igual)

Operadores lógicos:

- && (E lógico)
- || (OU lógico)
- ~ (NÃO lógico)

Exemplo com operadores lógicos:

```
% Verificar se um número está em um intervalo
x = 5;
if x >= 0 && x <= 10
    disp("x está entre 0 e 10");
else
    disp("x está fora do intervalo");
end
```

Estrutura **for** (repetição com número fixo de iterações):

```
% Exemplo 1: Somar números de 1 a 5
s = 0;
for k = 1:5
    s = s + k; % Adiciona k à soma
    fprintf("Iteração %d: soma = %d\n", k, s);
end
% Resultado final: s = 15

% Exemplo 2: Criar um vetor com quadrados
for i = 1:5
    quadrados(i) = i^2;
end
quadrados % Resultado: [1 4 9 16 25]
```

Variações do **for**:

```
% Com passo diferente de 1
for k = 0:0.5:2
    disp(k);
end
% Mostra: 0, 0.5, 1, 1.5, 2

% Iterar sobre elementos de um vetor
v = [10 20 30 40];
for elemento = v
    disp(elemento);
end
% Mostra cada elemento do vetor
```

Estrutura **while** (repetição com condição):

```
% Exemplo 1: Encontrar a primeira potência de 2 maior que 100
n = 1;
while n < 100
    n = 2 * n; % Multiplica n por 2
    fprintf("n = %d\n", n);
end
% Resultado: n = 128

% Exemplo 2: Contar até que o usuário digite 0
soma = 0;
while true
    x = input("Digite um número (0 para sair): ");
    if x == 0
        break; % Sai do loop
    end
    soma = soma + x;
end
disp(soma)
```

```

    end
    soma = soma + x;
end
fprintf("Soma total: %d\n", soma);

```

Comandos úteis em loops:

- `break` — sai do loop imediatamente
- `continue` — pula para a próxima iteração

2.4. Vetores e matrizes

Criação de vetores:

```

% Vetor linha (elementos separados por espaço)

v = [1 2 3 4];
% Resultado: v = [1 2 3 4]

% Vetor coluna (elementos separados por ponto-e-vírgula)

w = [1; 2; 3; 4];
% Resultado: w = [1]
%              [2]
%              [3]
%              [4]

% Vetor com sequência (de 0 a 1, passo 0.1)

t = 0:0.1:1;
% Resultado: [0 0.1 0.2 0.3 ... 0.9 1.0]

% Vetor com espaçamento uniforme (11 pontos entre 0 e 1)

u = linspace(0, 1, 11);
% Resultado: [0 0.1 0.2 0.3 ... 0.9 1.0]

```

Criação de matrizes:

```

% Matriz 2x2

A = [1 2; 3 4];
% Resultado: A = [1 2]
%              [3 4]

% Matriz 3x3

B = [1 2 3; 4 5 6; 7 8 9];

% Matriz de zeros (3x4)

Z = zeros(3, 4);

```

```
% Matriz de uns (2x3)
```

```
U = ones(2, 3);
```

```
% Matriz identidade (4x4)
```

```
I = eye(4);
```

```
% Matriz aleatória (2x3)
```

```
R = rand(2, 3); % Valores entre 0 e 1
```

Tamanho e dimensões:

```
v = [1 2 3 4];
```

```
length(v)          % Resultado: 4 (número de elementos)
```

```
A = [1 2; 3 4; 5 6];
```

```
size(A)             % Resultado: 3 2 (3 linhas, 2 colunas)
```

```
size(A, 1)          % Resultado: 3 (número de linhas)
```

```
size(A, 2)          % Resultado: 2 (número de colunas)
```


3. Scripts e Funções

3.1. Scripts (.m)

Um **script** é um arquivo de texto com extensão **.m** que contém uma sequência de comandos Octave. Útil para automatizar tarefas repetitivas.

Como criar um script:

- Abra um editor de texto (ou use o editor do Octave).
- Escreva os comandos.
- Salve como nome_do_script.m.
- Execute digitando nome_do_script no Octave.

Exemplo `aula01_script.m`:

```
% aula01_script.m
% Limpa a tela e as variáveis
clear;          % Remove todas as variáveis da memória
clc;           % Limpa a tela (Command Window)

% Cria um vetor x de 0 a 2π com 100 pontos
x = linspace(0, 2*pi, 100);

% Calcula y = sin(x) para cada valor de x
y = sin(x);

% Cria o gráfico
plot(x, y, 'b-', 'LineWidth', 2); % Linha azul, espessura 2

% Adiciona grade
grid on;

% Adiciona títulos e rótulos
title("Gráfico da função seno", 'FontSize', 14);
xlabel("x (radianos)", 'FontSize', 12);
ylabel("sin(x)", 'FontSize', 12);

% Adiciona legenda
legend("sin(x)");

% Define os limites dos eixos
xlim([0 2*pi]);
ylim([-1.5 1.5]);
```

Execute digitando no Octave:

```
aula01_script
```

3.2. Funções (modularização)

Uma **função** é um bloco de código reutilizável que recebe entrada (argumentos) e retorna saída (resultado).

Sintaxe básica:

```
function [saida1, saida2, ...] = nome_funcao(entrada1, entrada2,
...)
% Comentário descrevendo a função
% Corpo da função
saida1 = ...;
saida2 = ...;
end
```

Exemplo 1: Função simples

```
% Arquivo: area_circulo.m
function A = area_circulo(r)
% Calcula a área de um círculo
% Entrada: r (raio)
% Saída: A (área)
A = pi * r^2;
end
```

Teste:

```
area_circulo(2)           % Resultado: 12.5664
area_circulo([1 2 3])    % Resultado: [3.1416 12.5664 28.2743]
```

Exemplo 2: Função com múltiplas entradas e saídas (ivo: cilindro.m)

```
% Arquivo: cilindro.m
function [volume, area_lateral] = cilindro(r, h)
% Calcula volume e área lateral de um cilindro
% Entrada: r (raio), h (altura)
% Saída: volume, area_lateral

volume = pi * r^2 * h;
area_lateral = 2 * pi * r * h;
end
```

Uso:

```
[V, A] = cilindro(2, 5);
fprintf("Volume: %.2f\n", V);           % Resultado: 62.83
fprintf("Área lateral: %.2f\n", A);     % Resultado: 62.83
```

Exemplo 3: Função com validação

```
% Arquivo: raiz_quadrada.m
function r = raiz_quadrada(x)
    % Calcula a raiz quadrada com validação
    % Entrada: x (número)
    % Saída: r (raiz quadrada)

    if x < 0
        error("Erro: não é possível calcular raiz de número negativo!");
    end

    r = sqrt(x);
end
```

Uso:

```
raiz_quadrada(9)           % Resultado: 3
raiz_quadrada(-4)          % Erro: Erro: não é possível calcular raiz de
número negativo!
```

Dica importante: Operações elemento a elemento em funções

```
% Arquivo: quadrado_mais_um.m
function y = quadrado_mais_um(x)
    % Calcula  $y = x^2 + 1$  para cada elemento de x
    % Use .^ para elevar elemento a elemento
    y = x.^2 + 1;
end
```

Teste:

```
quadrado_mais_um(3)           % Resultado: 10
quadrado_mais_um([1 2 3])     % Resultado: [2 5 10]
```

4. Operações matriciais e vetoriais no Octave

4.1. Transposta, multiplicação, inversão

Transposta (troca linhas por colunas):

```
A = [1 2 3; 4 5 6];  
% A = [1 2 3]  
%      [4 5 6]  
  
AT = A'  
% AT = [1 4]  
%      [2 5]  
%      [3 6]
```

Multiplicação matricial (produto de matrizes):

```
A = [2 3; 4 5];  
B = [1 0; 0 1];  
  
C = A * B  
% Resultado: C = [2 3]  
%              [4 5]  
% (B é a matriz identidade, então C = A)
```

Resolução de sistemas lineares com A\b (recomendado):

```
% Sistema: 2x + 3y = 10  
%           4x + 5y = 18  
  
A = [2 3; 4 5];      % Matriz de coeficientes  
b = [10; 18];        % Vetor de constantes  
  
x = A\b              % Resolve Ax = b  
% Resultado: x = [2]  
%              [2]  
% Ou seja: x = 2, y = 2
```

Verificação:

```
% Verifica se a solução está correta  
residuo = A*x - b;  
fprintf("Resíduo: %.10f\n", norm(residuo));  
% Se o resíduo é próximo de zero, a solução está correta
```

Inversão de matriz (menos recomendado em cálculo numérico):

```
A = [2 3; 4 5];  
A_inv = inv(A);      % Calcula a inversa de A  
  
% Resolver usando inversa (não recomendado)  
x = inv(A) * b;      % Menos estável numericamente  
  
% Melhor usar:  
x = A \ b;           % Mais estável e rápido
```

Por que $A \backslash b$ é melhor que $\text{inv}(A) * b$?

- $A \backslash b$ usa decomposição LU (mais estável).
- $\text{inv}(A) * b$ calcula a inversa explicitamente (mais lento e menos preciso).

4.2. Indexação e slicing

Acessar elementos individuais:

```
A = [1 2 3; 4 5 6; 7 8 9];

A(1, 2)      % Linha 1, coluna 2 → Resultado: 2
A(3, 3)      % Linha 3, coluna 3 → Resultado: 9
A(2, 1)      % Linha 2, coluna 1 → Resultado: 4
```

Acessar linhas e colunas inteiras:

```
A = [1 2 3; 4 5 6; 7 8 9];

A(:, 2)      % Todas as linhas, coluna 2 → [2; 5; 8]
A(2, :)      % Linha 2, todas as colunas → [4 5 6]
A(:, :)      % Toda a matriz
```

Acessar submatrizes:

```
A = [1 2 3; 4 5 6; 7 8 9];

A(1:2, 1:2)  % Linhas 1-2, colunas 1-2 → [1 2; 4 5]
A(2:3, 2:3)  % Linhas 2-3, colunas 2-3 → [5 6; 8 9]
```

Vetores:

```
v = [10 20 30 40 50];

v(2)         % Elemento 2 → 20
v(1:3)       % Elementos 1 a 3 → [10 20 30]
v(2:end)     % Elementos 2 até o final → [20 30 40 50]
v(end)       % Último elemento → 50
```

4.3. Operações elemento a elemento

Diferença entre operação matricial e elemento a elemento:

```
x = [1 2 3];
y = [4 5 6];

% Operação elemento a elemento (com ponto)
z1 = x .* y      % Resultado: [4 10 18]
z2 = x ./ y      % Resultado: [0.25 0.4 0.5]
z3 = x .^ 2      % Resultado: [1 4 9]

% Operação matricial (sem ponto)
% x * y' seria produto escalar (não funciona com vetores linha)
```

Exemplo prático: Calcular seno de múltiplos ângulos

```
% Criar um vetor de ângulos (em radianos)
angulos = [0 pi/4 pi/2 pi];

% Calcular seno de cada ângulo (elemento a elemento)
senos = sin(angulos);
% Resultado: [0 0.7071 1 0 (aproximadamente)]

% Calcular cosseno de cada ângulo
cossenos = cos(angulos);
% Resultado: [1 0.7071 0 -1 (aproximadamente)]
```

Comparação: Operação matricial vs. elemento a elemento

```
A = [1 2; 3 4];
B = [5 6; 7 8];

% Produto matricial (sem ponto)
C1 = A * B
% C1 = [19 22]
%      [43 50]

% Produto elemento a elemento (com ponto)
C2 = A .* B
% C2 = [5 12]
%      [21 32]
```

5. Gráficos 2D e 3D (visualização de resultados)

5.1. Gráficos 2D - Plotar funções e dados

Gráfico simples:

```
% Criar dados
x = 0:0.01:2*pi;          % Vetor de 0 a 2π com passo 0.01
y = sin(x);               % Calcular seno de cada x

% Plotar
plot(x, y);               % Cria o gráfico
grid on;                  % Adiciona grade
title("Gráfico de sin(x)"); % Título
xlabel("x (radianos)");   % Rótulo do eixo x
ylabel("sin(x)");         % Rótulo do eixo y
```

Múltiplas curvas no mesmo gráfico:

```
x = 0:0.01:2*pi;
% Plotar duas funções
plot(x, sin(x), 'b', x, cos(x), 'r--');
% 'b' = azul (blue)
% 'r--' = vermelho (red) com linha tracejada (dashed)

grid on;
legend("sin(x)", "cos(x)"); % Legenda
title("Seno e Cosseno");
xlabel("x (radianos)");
ylabel("Valor");
```

Estilos de linha e cores:

```
% Cores: 'b' (azul), 'r' (vermelho), 'g' (verde), 'k' (preto), 'c' (ciano), 'm' (magenta)
% Linhas: '-' (sólida), '--' (tracejada), ':' (pontilhada), '-.' (traço-ponto)
% Marcadores: 'o' (círculo), 's' (quadrado), '^' (triângulo), '*' (estrela)

plot(x, sin(x), 'b-', 'LineWidth', 2); % Azul, sólida, espessura 2
plot(x, cos(x), 'r--', 'LineWidth', 1.5); % Vermelho, tracejada
plot(x, tan(x), 'g:', 'LineWidth', 1); % Verde, pontilhada
```

Subplots (múltiplos gráficos em uma figura):

```
x = 0:0.01:2*pi;

% Criar figura com 2 linhas e 1 coluna (2 gráficos)
subplot(2, 1, 1);
plot(x, sin(x), 'b');
grid on;
title("Seno");
ylabel("sin(x)");

subplot(2, 1, 2);
plot(x, cos(x), 'r');
grid on;
title("Cosseno");
xlabel("x (radianos)");
ylabel("cos(x)");
```

Controle de eixos:

```
x = 0:0.01:2*pi;
y = sin(x);

plot(x, y);
xlim([0 2*pi]);      % Define limites do eixo x
ylim([-1.5 1.5]);    % Define limites do eixo y
grid on;
```

5.2. Gráficos 3D — Visualizar superfícies

Gráfico de superfície (surf):

```
% Criar malha de pontos (x, y)
[x, y] = meshgrid(-2:0.1:2, -2:0.1:2);
% meshgrid cria duas matrizes: uma com valores de x, outra com
valores de y

% Calcular  $z = x^2 + y^2$  para cada ponto (x, y)
z = x.^2 + y.^2;

% Plotar a superfície
surf(x, y, z);
title("Parabolóide:  $z = x^2 + y^2$ ");
xlabel("x");
ylabel("y");
zlabel("z");
colorbar; % Adiciona barra de cores
```

Gráfico de malha (mesh):

```
[x, y] = meshgrid(-2:0.1:2, -2:0.1:2);
z = sin(x) .* cos(y);

mesh(x, y, z);
title("Superfície:  $z = \sin(x) \cdot \cos(y)$ ");
xlabel("x");
ylabel("y");
zlabel("z");
```

Gráfico de contorno (contour):

```
[x, y] = meshgrid(-2:0.1:2, -2:0.1:2);
z = x.^2 + y.^2;

% Contorno 2D (vista de cima)
contour(x, y, z, 10); % 10 = número de linhas de contorno
colorbar;
title("Contorno de  $z = x^2 + y^2$ ");
xlabel("x");
ylabel("y");
```


6.Representação numérica (binário, hexadecimal e decimal)

6.1. Conversões (binário/hex/decimal) no Octave

Decimal → Binário e Hexadecimal:

```
% Converter 13 (decimal) para binário
dec2bin(13)
% Resultado: '1101'

% Converter 255 (decimal) para hexadecimal
dec2hex(255)
% Resultado: 'FF'

% Converter 29 (decimal) para binário
dec2bin(29)
% Resultado: '11101'
```

Binário/Hexadecimal → Decimal:

```
% Converter '1101' (binário) para decimal
bin2dec("1101")
% Resultado: 13

% Converter 'FF' (hexadecimal) para decimal
hex2dec("FF")
% Resultado: 255

% Converter '7B' (hexadecimal) para decimal
hex2dec("7B")
% Resultado: 123
```

6.2. Operações com números binários (inteiro)

Soma binária:

```
% Exemplo: 13 + 7 em binário
% 13 = 1101
% 7 = 0111
% Soma:
%   1101
% + 0111
% -----
% 10100 (20 em decimal)

% Verificação no Octave:
resultado = bin2dec("1101") + bin2dec("0111");
fprintf("Resultado: %d\n", resultado); % Resultado: 20
fprintf("Em binário: %s\n", dec2bin(20)); % Em binário: 10100
```

Subtração binária:

```
% Exemplo: 13 - 7 em binário
% 13 = 1101
% 7 = 0111
% Subtração:
%   1101
% - 0111
% -----
%   0110 (6 em decimal)

resultado = bin2dec("1101") - bin2dec("0111");
fprintf("Resultado: %d\n", resultado); % Resultado: 6
fprintf("Em binário: %s\n", dec2bin(6)); % Em binário: 110
```

Multiplicação binária:

```
% Exemplo: 5 × 3 em binário
% 5 = 101
% 3 = 011
% Multiplicação:
%   101
% × 011
% -----
%   101
%  101
% 101
% -----
% 01111 (15 em decimal)

resultado = bin2dec("101") * bin2dec("011");
fprintf("Resultado: %d\n", resultado); % Resultado: 15
fprintf("Em binário: %s\n", dec2bin(15)); % Em binário: 1111
```

6.3. Binário “com parte decimal” (frações binárias)

Um número decimal com parte fracionária pode ser representado em binário usando potências negativas de 2:

$$0.625_{10} = 0.101_2 \text{ porque: } 0.101_2 = 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} = 0.5 + 0 + 0.125 = 0.625$$

No Octave (verificação):

$$0.5 + 0.125$$

Verificação no Octave:

```
% Calcular 0.5 + 0.125
resultado = 0.5 + 0.125;
fprintf("0.5 + 0.125 = %.3f\n", resultado); % Resultado: 0.625

% Outro exemplo: 0.75 = 0.11 em binário
% 0.11 = 1 · 2-1 + 1 · 2-2 = 0.5 + 0.25 = 0.75
resultado2 = 0.5 + 0.25;
fprintf("0.5 + 0.25 = %.2f\n", resultado2); % Resultado: 0.75
```

Ponto-chave para erros numéricos:

Muitos decimais "simples" em base 10 não têm representação finita em binário. Por exemplo, 0.1 não pode ser representado exatamente em binário com um número finito de dígitos.

Teste clássico de erro de arredondamento:

```
% Mostrar mais casas decimais
format long

% Calcular 0.1 + 0.2
resultado = 0.1 + 0.2;
fprintf("0.1 + 0.2 = %.20f\n", resultado);
% Resultado: 0.300000000000000004

% Comparar com 0.3
valor_esperado = 0.3;
fprintf("0.3 = %.20f\n", valor_esperado);
% Resultado: 0.299999999999999999

% Verificar se são iguais
if resultado == valor_esperado
    disp("São iguais");
else
    disp("São diferentes!"); % Isso será exibido
end
```

Comparação correta com tolerância:

```
% Usar tolerância (epsilon) para comparação
tolerancia = 1e-12; % 10-12

if abs((0.1 + 0.2) - 0.3) < tolerancia
    disp("São iguais (dentro da tolerância)");
else
    disp("São diferentes");
end

% Mostrar o erro
erro = abs((0.1 + 0.2) - 0.3);
fprintf("Erro: %.20e\n", erro); % Mostra em notação científica
```

6.4. Complemento de 2 (inteiros com sinal)

Conceito:

Em 8 bits, o intervalo de números representáveis é:

- Sem sinal: 0 a 255
- Com sinal (complemento de 2): -128 a 127

Como representar um número negativo em complemento de 2:

Exemplo: Representar -5 em 8 bits

1. Escrever +5 em binário: 00000101
2. Inverter todos os bits: 11111010
3. Somar 1: 11111011

Resultado: **-5 em complemento de 2 = 11111011**

Verificação no Octave:

```
% Criar inteiros de 8 bits
```

```

a = int8(5);
b = int8(-5);

fprintf("5 em int8: %d\n", a);    % Resultado: 5
fprintf("-5 em int8: %d\n", b);    % Resultado: -5

% Mostrar representação binária (conceitual)
% Octave não tem função nativa para mostrar binário de int8,
% mas podemos verificar o comportamento

```

Overflow em complemento de 2:

```

% Em int8, o máximo é 127
max_int8 = int8(127);
fprintf("Máximo int8: %d\n", max_int8);    % Resultado: 127

% Adicionar 1 causa overflow
resultado = int8(127) + int8(1);
fprintf("127 + 1 em int8: %d\n", resultado);    % Resultado: -128
(overflow!)

% Isso ocorre porque:
% 127 = 01111111
% 1   = 00000001
% Soma = 10000000 (interpretado como -128 em complemento de 2)

```

Outro exemplo: Subtração com overflow

```

% Mínimo int8 é -128
min_int8 = int8(-128);
fprintf("Mínimo int8: %d\n", min_int8);    % Resultado: -128

% Subtrair 1 causa overflow
resultado = int8(-128) - int8(1);
fprintf("-128 - 1 em int8: %d\n", resultado);    % Resultado: 127
(overflow!)

```

6.5. Tipos inteiros: int8, int16, int32, int64

Intervalos de representação:

TIPO	BITS	MÍNIMO	MÁXIMO
INT8	8	-128	127
INT16	16	-32,768	32,767
INT32	32	-2,147,483,648	2,147,483,647
INT64	64	-9,223,372,036,854,775,808	9,223,372,036,854,775,807

Exemplos no Octave:

```
% int8: intervalo pequeno
a = int8(100);
b = int8(30);
resultado = a + b;
fprintf("int8: 100 + 30 = %d (overflow!)\n", resultado);
% Resultado: -126 (porque 130 > 127)

% int16: intervalo maior
a = int16(100);
b = int16(30);
resultado = a + b;
fprintf("int16: 100 + 30 = %d (ok)\n", resultado);
% Resultado: 130

% int32 e int64: intervalos muito maiores
a = int32(1000000);
b = int32(2000000);
resultado = a + b;
fprintf("int32: 1000000 + 2000000 = %d\n", resultado);
% Resultado: 3000000

% Verificar o tipo de uma variável
class(3)           % Resultado: 'double'
class(int64(3))    % Resultado: 'int64'
```

6.6. Representação de números decimais (ponto flutuante)

Padrão IEEE 754 (double precision):

Um número em ponto flutuante é representado como:

$$\text{número} = \text{senal} \times \text{mantissa} \times 2^{\text{expoente}}$$

Exemplo:

```
% Número: 5.25
% Representação: 5.25 = 1.3125 × 2^2
% Sinal: positivo
% Mantissa: 1.3125
% Expoente: 2

% Verificação
resultado = 1.3125 * (2^2);
fprintf("1.3125 × 2^2 = %.2f\n", resultado); % Resultado: 5.25
```

Precisão e limites:

```
% Maior número representável em double
realmax
% Resultado: 1.7977e+308

% Menor número positivo representável
realmin
% Resultado: 2.2251e-308

% Precisão de máquina (epsilon)
eps
% Resultado: 2.2204e-16
% Significa: diferença mínima entre 1 e o próximo número
representável
```

Erros de arredondamento em operações:

```
format long

% Exemplo 1: Perda de precisão
a = 1e16;
b = 1;
resultado = (a + b) - a;
fprintf("(10^16 + 1) - 10^16 = %.1f\n", resultado);
% Resultado esperado: 1
% Resultado real: 0 (porque 1 é muito pequeno comparado a 10^16)

% Exemplo 2: Cancelamento catastrófico
x = 0.1;
y = 0.1 + 1e-15;
resultado = (y - x) / 1e-15;
fprintf("Resultado: %.10f\n", resultado);
% Pode não ser exatamente 1 devido a erros de arredondamento
```

6.7. Tabela ASCII — exemplos essenciais

A tabela ASCII associa caracteres (letras, números, símbolos) a códigos numéricos (0–127).

Exemplos comuns:

- '0' → 48
- '9' → 57
- 'A' → 65
- 'Z' → 90
- 'a' → 97
- 'z' → 122

Obter código ASCII de um caractere no Octave:

```
% Função: double() converte caractere para código ASCII
double('A')      % Resultado: 65
double('Z')      % Resultado: 90
double('a')      % Resultado: 97
double('z')      % Resultado: 122
double('0')      % Resultado: 48
double('9')      % Resultado: 57
double(' ')      % Resultado: 32 (espaço)
```

Obter caractere a partir do código ASCII:

```
% Função: char() converte código ASCII para caractere
char(65)         % Resultado: 'A'
char(90)         % Resultado: 'Z'
char(97)         % Resultado: 'a'
char(122)        % Resultado: 'z'
char(48)         % Resultado: '0'
char(57)         % Resultado: '9'
char(32)         % Resultado: ' ' (espaço)
```

Gerando um trecho útil da tabela (32 a 126):

```
% Script: tabela_ascii.m
clear; clc;

% Criar vetor de códigos (32 a 126)
% 32 = espaço (primeiro caractere imprimível)
% 126 = til (último caractere imprimível)
codigos = 32:126;

% Converter códigos para caracteres
caracteres = char(codigos);

% Exibir tabela
fprintf("Código | Caractere\n");
fprintf("-----|-----\n");
for i = 1:length(codigos)
    fprintf("%3d   | %c\n", codigos(i), caracteres(i));
end
```

7. Representação simbólica de equações no Octave

7.1. Introdução ao pacote symbolic

O Octave permite trabalhar com equações simbólicas usando o pacote symbolic (compatível com SymPy do Python).

Instalação (primeira vez):

```
% No Octave, execute:  
pkg install -forge symbolic  
pkg load symbolic
```

Carregar o pacote em cada sessão:

```
pkg load symbolic
```

7.2. Criar variáveis simbólicas

Definir variáveis simbólicas:

```
pkg load symbolic  
  
% Criar variável simbólica x  
syms x  
  
% Criar múltiplas variáveis  
syms a b c  
  
% Criar variável com propriedades (ex: real)  
syms x real
```

7.3. Criar e manipular equações simbólicas

Exemplo 1: Equação quadrática

```
pkg load symbolic  
syms x  
  
% Definir a equação:  $x^2 - 5x + 6 = 0$   
eq = x^2 - 5*x + 6;  
  
% Exibir a equação  
disp(eq);  
% Resultado:  $x^2 - 5x + 6$   
  
% Resolver a equação  
solucoes = solve(eq, x);  
disp(solucoes);  
% Resultado: [2; 3]
```


Exemplo 2: Sistema de equações lineares

```
pkg load symbolic
syms x y

% Definir o sistema:
% 2x + 3y = 10
% 4x + 5y = 18

eq1 = 2*x + 3*y - 10;
eq2 = 4*x + 5*y - 18;

% Resolver o sistema
solucao = solve([eq1, eq2], [x, y]);

% Exibir solução
fprintf("x = %s\n", solucao.x);
fprintf("y = %s\n", solucao.y);
% Resultado: x = 2, y = 2
```

Exemplo 3: Equação não-linear

```
pkg load symbolic
syms x

% Definir a equação: sin(x) = 0.5
eq = sin(x) - 0.5;

% Resolver (encontra uma solução)
solucao = solve(eq, x);
disp(solucão);
% Resultado: pi/6 (ou 30 graus)
```

7.4. Operações com expressões simbólicas

Simplificar expressões:

```
pkg load symbolic
syms x

% Expressão complexa
expr = (x^2 + 2*x + 1) / (x + 1);

% Simplificar
expr_simplificada = simplify(expr);
disp(expr_simplificada);
% Resultado: x + 1
```

Expandir expressões:

```
pkg load symbolic
syms x

% Expressão fatorada
expr = (x + 1)^3;

% Expandir
expr_expandida = expand(expr);
disp(expr_expandida);
% Resultado: x^3 + 3*x^2 + 3*x + 1
```

Fatorar expressões:

```
pkg load symbolic
syms x

% Expressão expandida
expr = x^2 - 5*x + 6;

% Fatorar
expr_fatorada = factor(expr);
disp(expr_fatorada);
% Resultado: (x - 2)*(x - 3)
```

7.5. Derivadas e integrais simbólicas

Calcular derivada:

```
pkg load symbolic
syms x

% Função: f(x) = x^3 + 2*x^2 - 5*x + 1
f = x^3 + 2*x^2 - 5*x + 1;

% Primeira derivada
f_prima = diff(f, x);
disp(f_prima);
% Resultado: 3*x^2 + 4*x - 5

% Segunda derivada
f_segunda = diff(f, x, 2);
disp(f_segunda);
% Resultado: 6*x + 4
```

Calcular integral:

```
pkg load symbolic
syms x

% Função: f(x) = x^2 + 3*x
f = x^2 + 3*x;

% Integral indefinida
F = int(f, x);
disp(F);
% Resultado: x^3/3 + 3*x^2/2

% Integral definida (de 0 a 2)
integral_definida = int(f, x, 0, 2);
disp(integral_definida);
% Resultado: 14/3 (aproximadamente 4.667)
```

7.6. Substituição de valores em expressões simbólicas

Usar **subs()** para substituir variáveis:

```
pkg load symbolic
syms x y

% Expressão:  $x^2 + 2xy + y^2$ 
expr = x^2 + 2*x*y + y^2;

% Substituir  $x = 3$ 
expr1 = subs(expr, x, 3);
disp(expr1);
% Resultado:  $9 + 6y + y^2$ 

% Substituir  $x = 3$  e  $y = 2$ 
expr2 = subs(expr, {x, y}, {3, 2});
disp(expr2);
% Resultado: 25

% Converter para número
resultado = double(expr2);
fprintf("Resultado: %d\n", resultado);
% Resultado: 25
```

8. Octave aplicado ao Cálculo Numérico

8.1. Sistemas lineares com $A \backslash b$ (exemplo da engenharia)

Sistema:

- $2x + 3y = 10$
- $4x + 5y = 18$

Em Octave:

```
% Definir a matriz de coeficientes
A = [2 3; 4 5];

% Definir o vetor de constantes
b = [10; 18];

% Resolver o sistema Ax = b
x = A \ b;

% Exibir a solução
fprintf("I1 = %.4f A\n", x(1)); % Resultado: 2.0000 A
fprintf("I2 = %.4f A\n", x(2)); % Resultado: 2.0000 A
```

Verificação da solução:

```
% Verificar: A*x deve ser igual a b
residuo = A*x - b;
fprintf("Resíduo: %.10e\n", norm(residuo));
% Se o resíduo é próximo de zero, a solução está correta
```

8.2 Exploração de condicionamento (sensibilidade a perturbações)

Conceito:

Um sistema "mal-condicionado" é sensível a pequenas mudanças nos dados.

Teste: Alterar levemente o vetor b

```
% Sistema original
A = [2 3; 4 5];
b1 = [10; 18];
x1 = A \ b1;
fprintf("Solução original: x1 = [%.4f, %.4f]\n", x1(1), x1(2));

% Sistema com perturbação pequena (0.1 em b1)
b2 = [10.1; 18];
x2 = A \ b2;
fprintf("Solução perturbada: x2 = [%.4f, %.4f]\n", x2(1), x2(2));

% Calcular a mudança relativa
mudanca_relativa = norm(x2 - x1) / norm(x1);
fprintf("Mudança relativa na solução: %.4f\n", mudanca_relativa);
```

Número de condição:

```
% Calcular o número de condição de A
cond_A = cond(A);
fprintf("Número de condição: %.4f\n", cond_A);
% Número de condição próximo de 1 = bem-condicionado
% Número de condição grande = mal-condicionado
```